Machine Learning and Pattern Recognition

Unsupervised Learning

Dr. Kewei "Isaac" Li

Department of Digital Engineering
Long Island University

April 25, 2025

Outline

- Introduction: What is Unsupervised learning?
- K-Means Clustering
- Hierarchical Clustering
- Density Estimation
- Autoencoders for Dimensionality Reduction

Introduction

- In supervised learning, our training data have both features/inputs and corresponding labels/outputs

input image

Label

← "dog"

input image

← "cat"

- We train our models so that it can predict labels for new data

Introduction

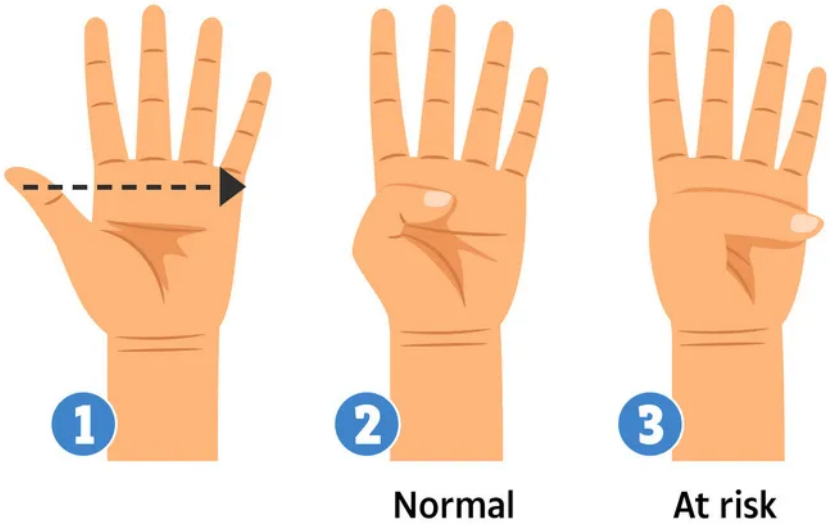
- In contrast to supervised learning, with unsupervised learning, we only have features but no corresponding labels for our data
- We are not interested in prediction, because we do not have an associated output variable
- On natural question: if there are no labels, what are we learning?

Introduction

Unsupervised Learning

- One example of unsupervised learning is clustering
- In clustering, the goal is to develop algorithms that can reason about "similarity" of data points, and group the data points into clusters.
- Autoencoders: we search for insights about our data by learning compressed versions of the original data.

Introduction



Thumb-palm test for detection of aortic aneurysm by Dr. John Elefteriades from Yale School of Medicine

Image: www.thesun.co.uk

Introduction

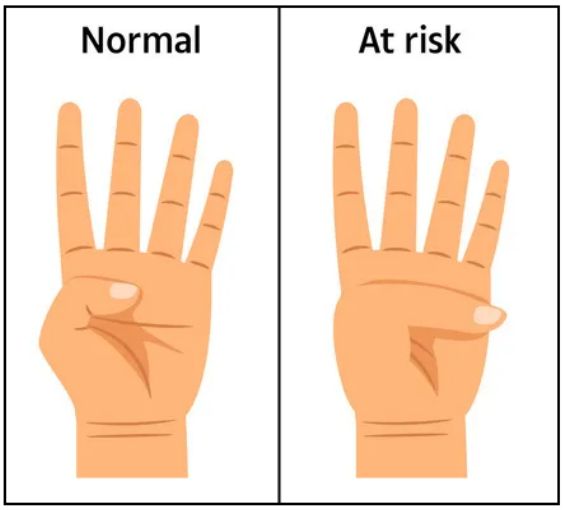


Image: www.thesun.co.uk

- We just finished an clustering task.
- We used the results from thumb-palm test
- Based on these results, we divided the population into two cohorts (clusters): normal and at risk

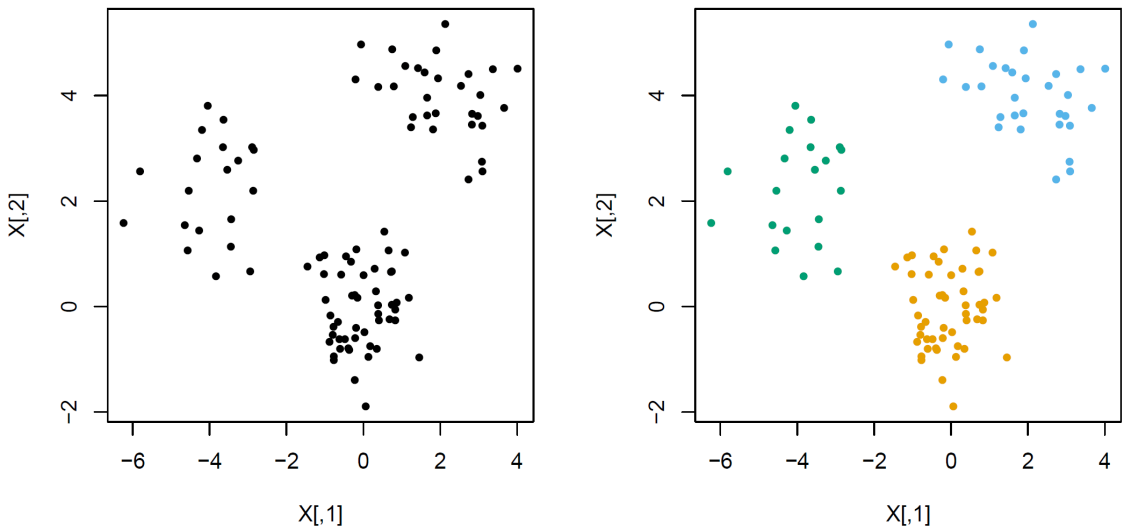
Outline

- Introduction: What is Unsupervised learning?
- K-Means Clustering
- Hierarchical Clustering
- Density Estimation
- Autoencoders for Dimensionality Reduction

Examples of Clustering Tasks

- Identify similar groups of online shoppers based on their browsing and purchasing history
- Identify similar groups of video viewers based on their ratings or recent listening/viewing patterns
- Cluster hospital patients based on their medical histories
- Determine how to place broadcasting towers centers to guarantee that desired coverage criteria are met

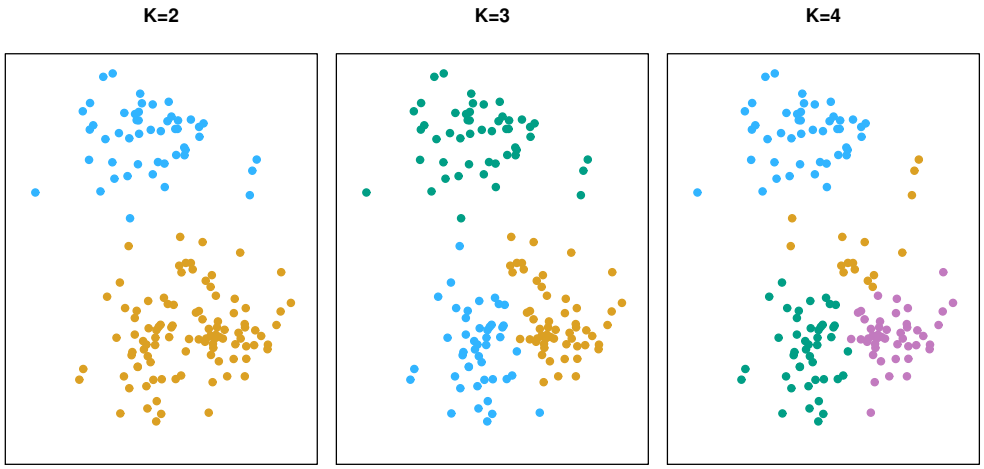
Examples of Clustering



Left: Data. Right: One possible way to cluster the data

Clustering

- Clustering is a partition $\{C_1, \ldots, C_K\}$ where each cluster C_k denotes a subset of the observations
- Each observation belongs to one and only one of the clusters
- To denote the observation $\mathbf{x}_i$ is in the k-th cluster, we write $\mathbf{x}_i \in C_k$



Clustering

- A good clustering is one for which the within-cluster variation (WCV) is as small as possible.
- The WCV for cluster C_k, denoted by $W(C_k)$, is a measure of the amount by which the observations in C_k differ from one another
- Goal of K-means clustering: Partition the observations $\{\mathbf{x}_i \in \mathbb{R}^p\}$ into K clusters $\{C_1, \ldots, C_K\}$ that minimize the total WCV,

$$\sum_{k=1}^{K} W(C_k)$$

Clustering

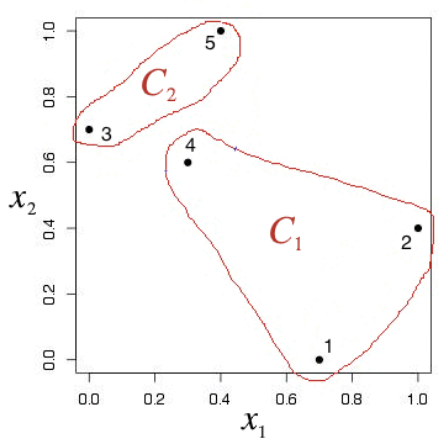
- Typically, we use Euclidean distance for WCV. For cluster C_k,

$$W(C_k) = \frac{1}{|C_k|} \sum_{i,i' \in C_k} \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2$$

$|C_k|$: the number of observations in cluster C_k
$x_i, x_{i'}$: two different observations in cluster C_k

Clustering

A numerical example with $n = 5$, $p = 2$ and $K = 2$



The full distance matrix is given below

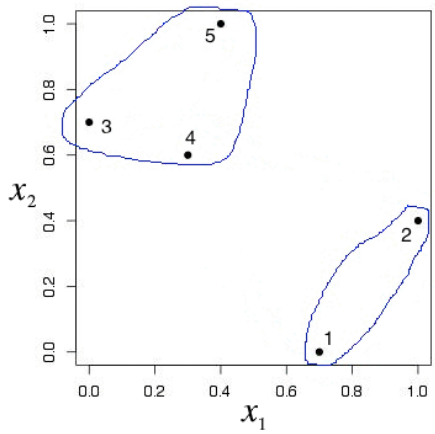
	1	2	3	4	5
1	0	0.25	0.98	0.52	1.09
2	0.25	0	1.09	0.53	0.72
3	0.98	1.09	0	0.10	0.25
4	0.52	0.53	0.10	0	0.17
5	1.09	0.72	0.25	0.17	0

- Red clustering:

$$\sum W(C_k) = W(C_1) + W(C_2) = \frac{0.25 + 0.53 + 0.52}{3} + \frac{0.25}{2} = 0.56$$

Clustering

A numerical example with $n = 5$, $p = 2$ and $K = 2$



The full distance matrix is given below

	1	2	3	4	5
1	0	0.25	0.98	0.52	1.09
2	0.25	0	1.09	0.53	0.72
3	0.98	1.09	0	0.10	0.25
4	0.52	0.53	0.10	0	0.17
5	1.09	0.72	0.25	0.17	0

- Blue clustering:

$$\sum W(C_k) = \frac{0.1 + 0.25 + 0.17}{3} + \frac{0.25}{2} = 0.30 \text{ (smaller)}$$

Clustering

- How do we minimize the total WCV?

$$\sum_{k=1}^{K} W(C_k) = \sum_{k=1}^{K} \frac{1}{|C_k|} \sum_{i,i' \in C_k} \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2$$

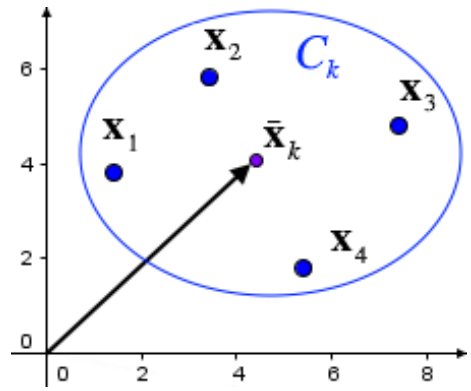
- It is computationally infeasible to actually minimize this equation
- We have to try all possible partitions of the n points into K clusters
- When $n = 25$ and $K = 4$, there are 5×10^{13} possible partitions
- We will have to settle for an approximate solution

K-means Clustering

- We rewrite WCV more conveniently as sum of squared distances (SSD):

$$W(C_k) = 2 \sum_{i \in C_k} ||\mathbf{x}_i - \bar{\mathbf{x}}_k||^2, \quad \bar{\mathbf{x}}_k = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}_i$$

- Here $\bar{\mathbf{x}}_k$ is the centroid of all points in cluster C_k

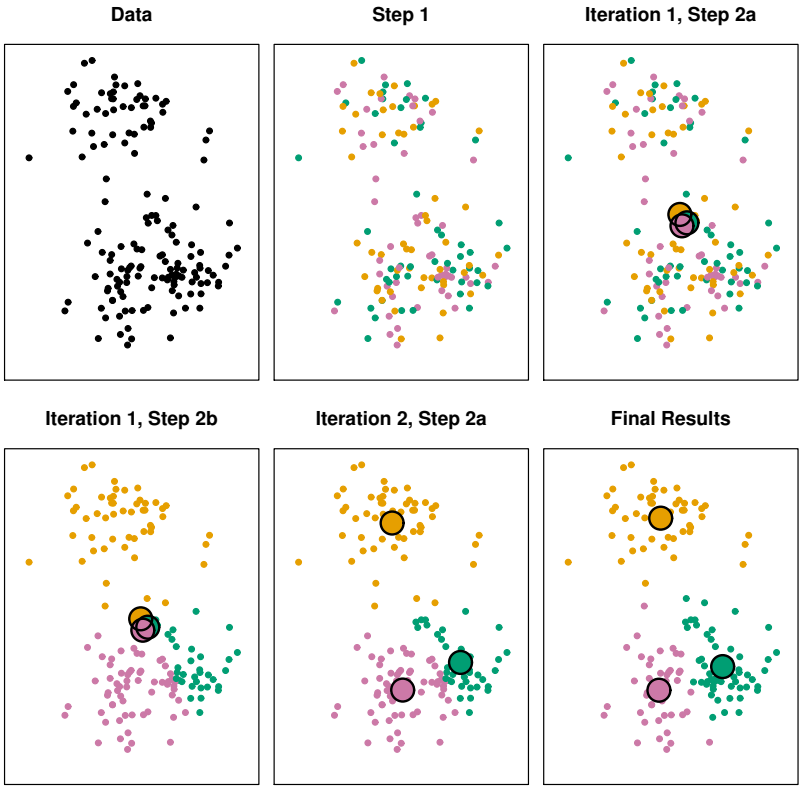


K-means Clustering

K-Means Clustering Algorithm

1. Randomly assign a number, from 1 to K, to each of the observations. These serve as initial cluster assignments for the observations
2. Iterate until the cluster assignments stop changing:
   a. For each of the K clusters, compute the cluster centroid.
   b. Re-assign each observation to the cluster whose centroid is closest

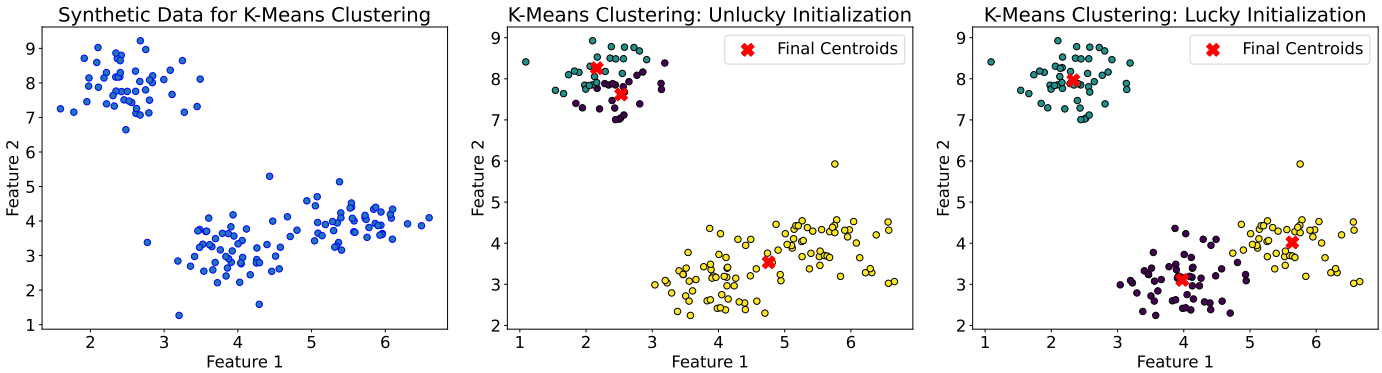
K-means Clustering



K-means Clustering

Importance of Initialization

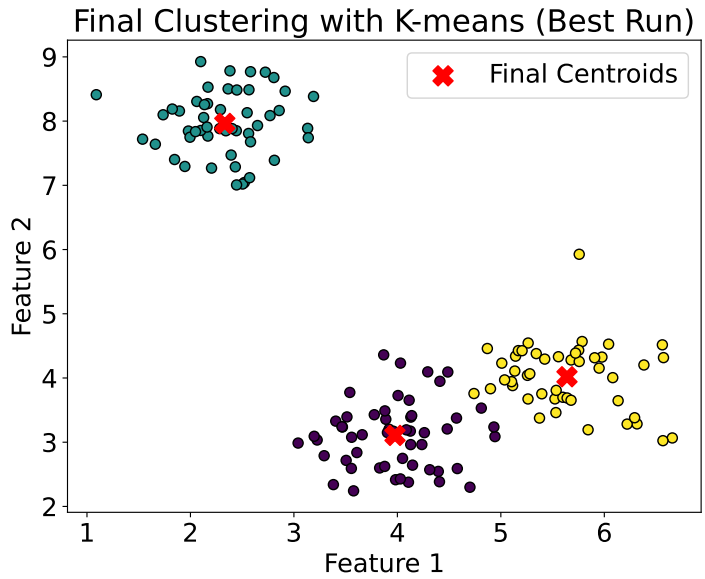
- The K-means algorithm is only guaranteed to converge to a local minimum, not necessarily a global minimum.
- Thus, the result depends on how we initialize the cluster means.



K-means Clustering

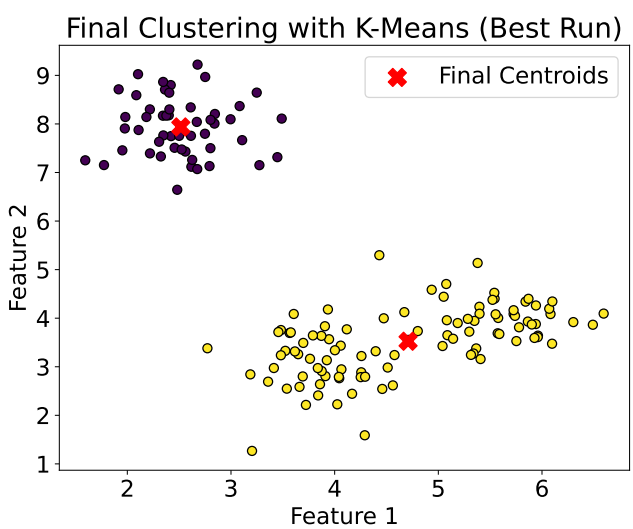
K-means++ algorithm

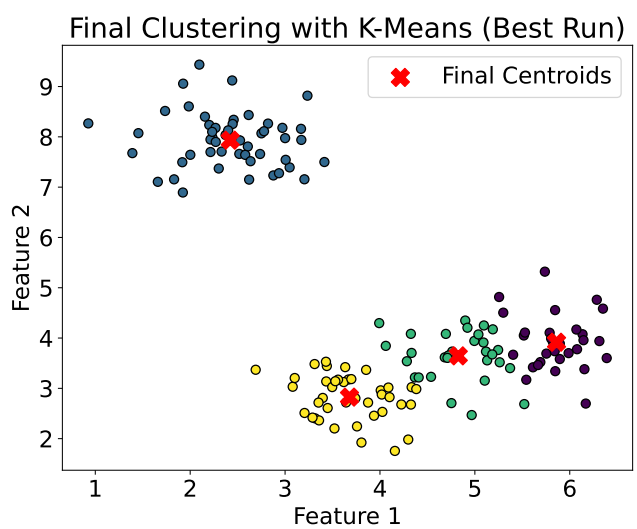
- Run the K-means algorithm multiple times with different initial conditions, and then pick the best run.



K-means Clustering

- An important choice in clustering is the number of clusters (K)
- Higher K means more clusters and lower within-cluster variance





K-means Clustering

Python Implementation

- Download code from Brightspace

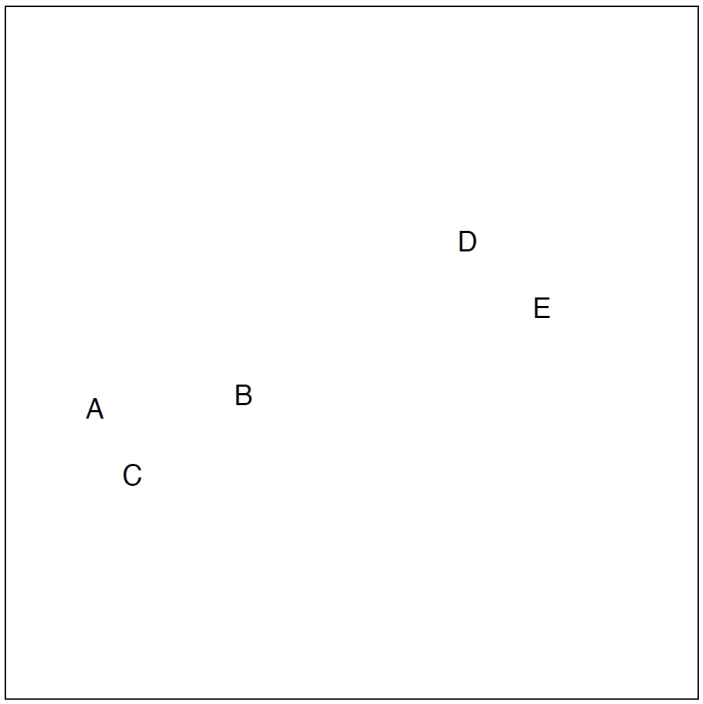
Outline

- Introduction: What is Unsupervised learning?
- K-Means Clustering
- Hierarchical Clustering
- Density Estimation
- Autoencoders for Dimensionality Reduction

Hierarchical Clustering

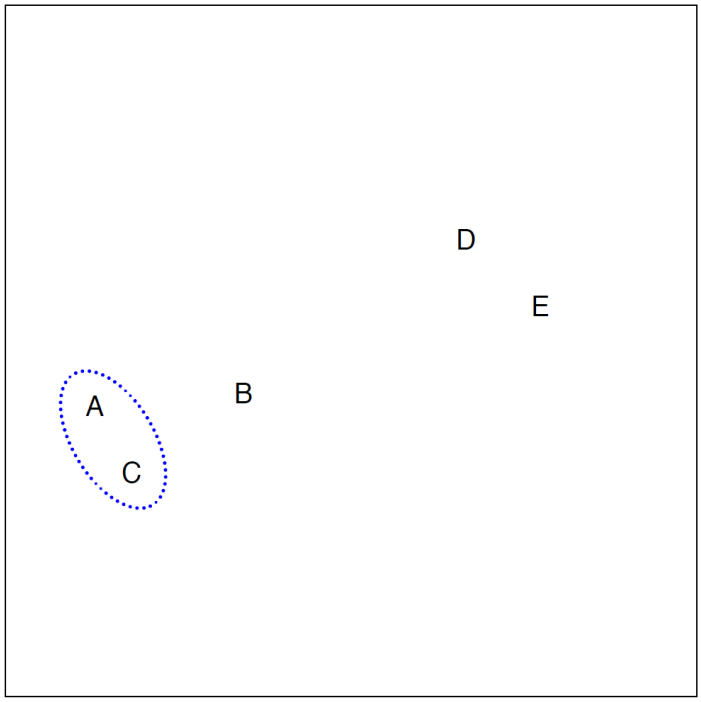
- K-means is an objective-based approach that requires user to specify the number of clusters K
- The result is somewhat random: it depends on the random initialization we started with
- Hierarchical clustering does not require a pre-specified choice of K, and which provides a deterministic answer
- We will focus on bottom-up hierarchical clustering

Hierarchical Clustering



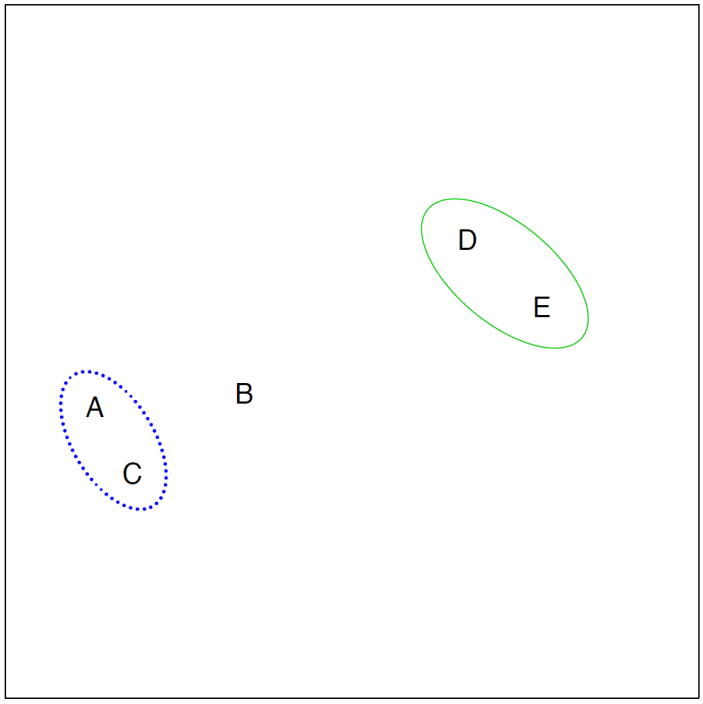
Each point starts as an individual cluster

Hierarchical Clustering



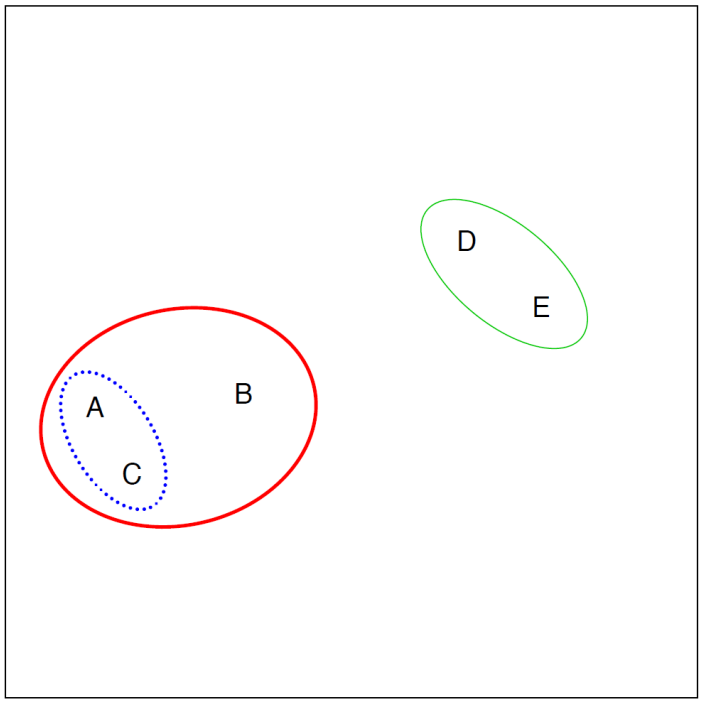
We merge the two closet clusters (points) into one

Hierarchical Clustering



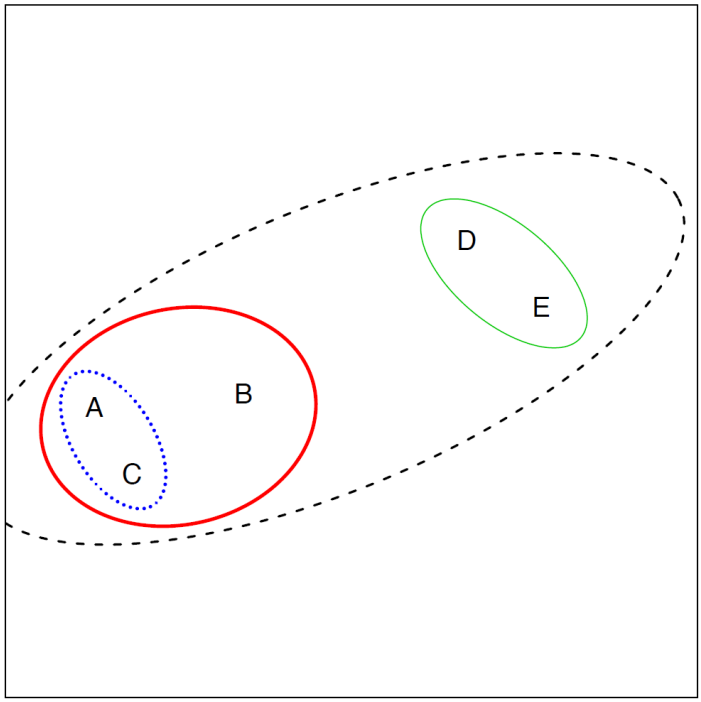
Then we merge the next two closest clusters

Hierarchical Clustering



Then the next two closest clusters ...

Hierarchical Clustering



Until at last all of the points are all in a single cluster

Hierarchical Clustering

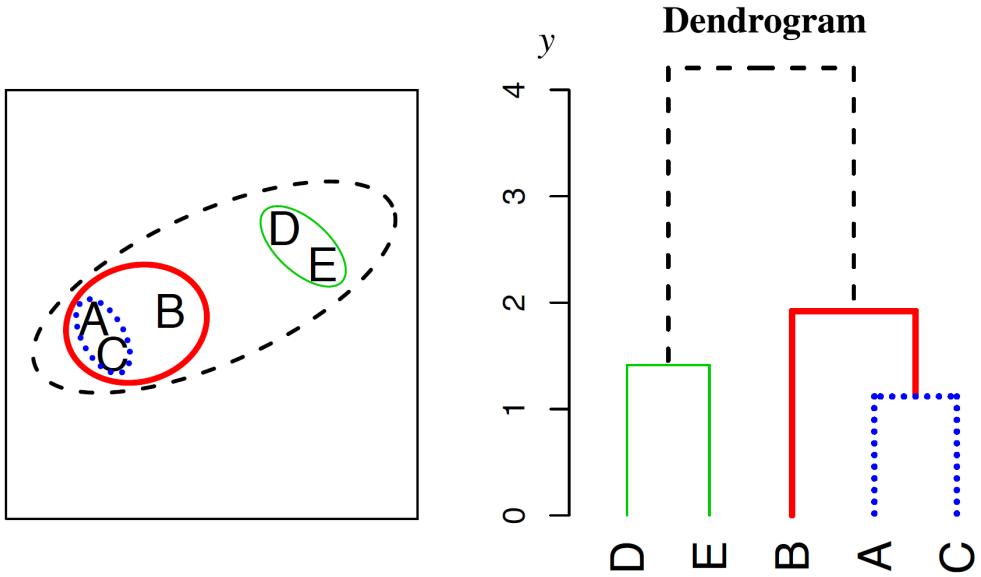
Bottom-up hierarchical clustering algorithm

- Start with each point in its own cluster
- Identify the two closest clusters. Merge them
- Repeat until all points are in a single cluster

Hierarchical Clustering

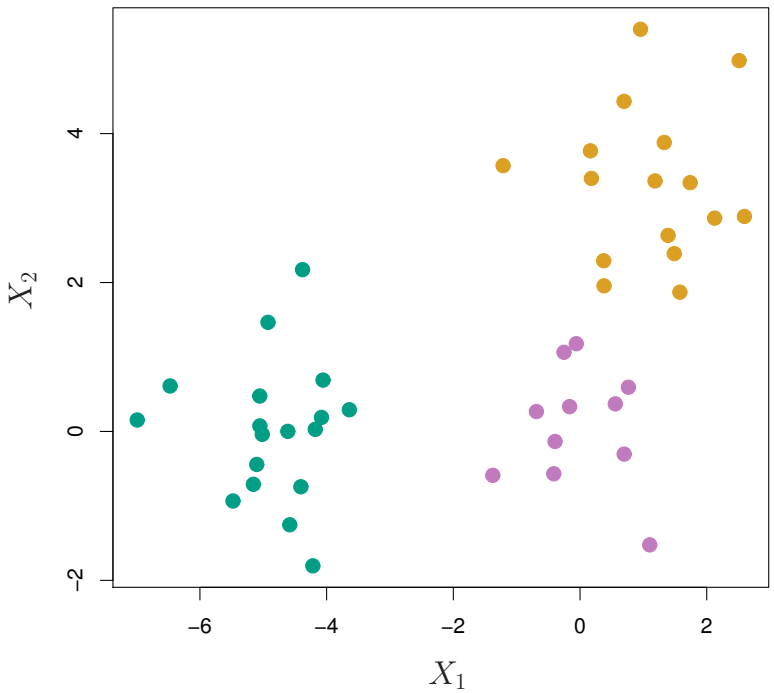
Bottom-up hierarchical clustering algorithm

At the end, we obtain a dendrogram. The y-axis is proportional to the distance between the clusters that got merged at that step



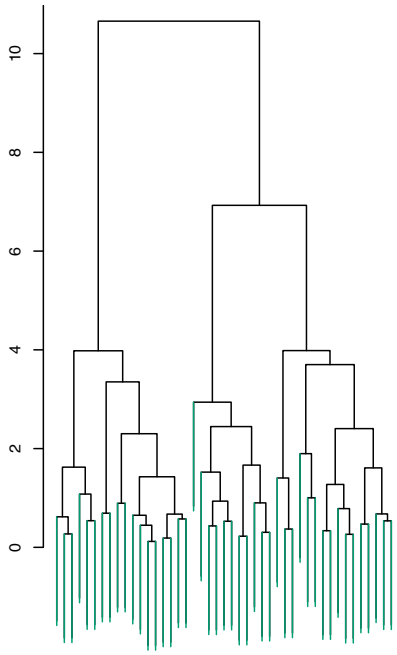
Hierarchical Clustering

Another example, $n = 45$ points



Hierarchical Clustering

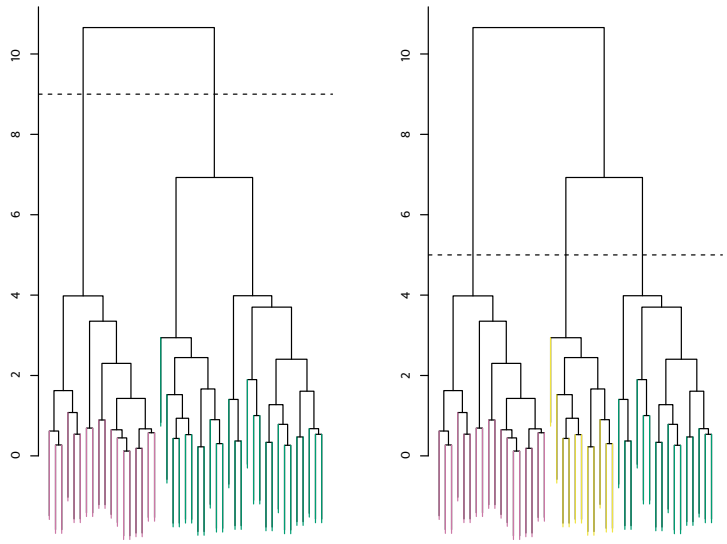
Dendrogram obtained from hierarchical clustering:



Hierarchical Clustering

Cutting dendrogram to obtain the clustering results

- Left: Dendrogram cut at height $y = 9$, resulting in $K = 2$ clusters
- Right: Dendrogram cut at height $y = 5$, resulting in $K = 3$ clusters



Outline

- Introduction: What is Unsupervised learning?
- K-Means Clustering
- Hierarchical Clustering
- Density Estimation
- Autoencoders for Dimensionality Reduction

Part II:
Density Estimation

Density Estimation

- Density Estimation is a core task in unsupervised learning
- It focuses on estimating the probability distribution from which the training data was drawn — without labeled data.
- Suppose we have a set of observations $x_1, x_2, \ldots, x_n$ drawn from an unknown distribution $p(x)$.
- The goal of density estimation is to estimate this $p(x)$ based only on the data.

Density Estimation

Applications:

- Understanding the structure of the data
- Identifying anomalies (outliers lie in low-density regions)
- Sampling new data points (with the estimated probability density)
- Preprocessing (e.g., feature transformations)

Density Estimation

Types of Density Estimation:

- Parametric Density Estimation: assume a specific form for the distribution (e.g., Gaussian) and estimate its parameters.
- Non-Parametric Density Estimation
  - Histogram: divide the range of data into bins and count the frequency.
  - Kernel Density Estimation (KDE)
  - k-Nearest Neighbors Density Estimation

Density Estimation

Kernel Density Estimation:

- It is a popular and powerful non-parametric method for estimating the probability density function of a random variable.
- It is used to smoothly estimate the underlying distribution of data points without assuming any specific distributional form (e.g. Gaussian)
- It's essentially a smoothed histogram using a kernel function.

Density Estimation

Kernel Density Estimation:

- Suppose we have a set of observations $x_1, x_2, \ldots, x_n$ drawn from an unknown distribution $p(x)$, we wish to estimate $p(x)$ for x_0
- A Kernel Density Estimate has the form

$$\hat{p}(x) = \frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right)$$

n : number of data points
h : bandwidth (controls smoothness)
$K(\cdot)$: kernel function (e.g., Gaussian)
x_i : data points

Density Estimation

Common Kernel Functions:

- The kernel $K(u)$ is a smooth, symmetric function that integrates to 1.
- Gaussian:

$$K(u) = \frac{1}{\sqrt{2\pi}} e^{-u^2/2}$$

- Epanechnikov:

$$K(u) = \frac{3}{4}(1 - u^2) \quad \text{if } |u| \leq 1, \text{ else } 0$$

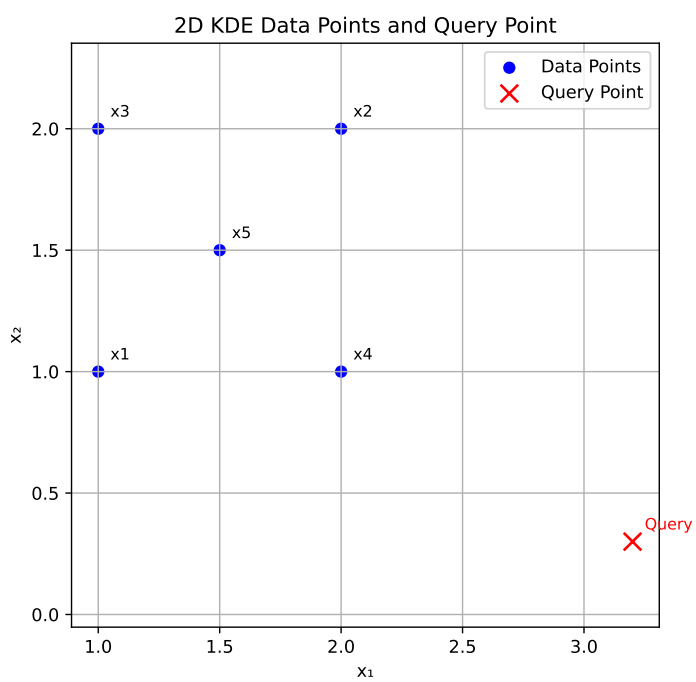
- Uniform:

$$K(u) = \frac{1}{2} \quad \text{if } |u| \leq 1, \text{ else } 0$$

- Most commonly, Gaussian kernels are used.

Density Estimation

An example: Estimate the density for the query point $\mathbf{x} = [3.2, 0.3]$



Density Estimation

An example:

- Compute squared distances by using

$$\|\mathbf{x} - \mathbf{x}_i\|^2 = (\mathbf{x}_1 - \mathbf{x})^2 + (\mathbf{x}_2 - \mathbf{x})^2 + \dots$$

- We'll calculate squared distance for each data point:

$$\begin{aligned}
\|\mathbf{x} - \mathbf{x}_1\|^2 &= (3.2 - 1)^2 + (0.3 - 1)^2 = 4.84 + 0.49 = 5.33 \\
\|\mathbf{x} - \mathbf{x}_2\|^2 &= (3.2 - 2)^2 + (0.3 - 2)^2 = 1.44 + 2.89 = 4.33 \\
\|\mathbf{x} - \mathbf{x}_3\|^2 &= (3.2 - 1)^2 + (0.3 - 2)^2 = 4.84 + 2.89 = 7.73 \\
\|\mathbf{x} - \mathbf{x}_4\|^2 &= (3.2 - 2)^2 + (0.3 - 1)^2 = 1.44 + 0.49 = 1.93 \\
\|\mathbf{x} - \mathbf{x}_5\|^2 &= (3.2 - 1.5)^2 + (0.3 - 1.5)^2 = 2.89 + 1.44 = 4.33
\end{aligned}$$

Density Estimation

An example:

- General Gaussian Kernel in 2D:

$$K(\mathbf{x}, \mathbf{x}_i) = \frac{1}{2\pi h^2} \exp\left(-\frac{1}{2h^2}\|\mathbf{x} - \mathbf{x}_i\|^2\right)$$

- With a bandwidth of $h = 0.5$, we can compute each kernel:

$$\begin{aligned}
K_1 &= \frac{1}{0.5\pi} \exp\left(-\frac{5.33}{0.5}\right) = 0.6366 \cdot e^{-10.66} \approx 1.48 \times 10^{-5} \\
K_2 &= 0.6366 \cdot e^{-8.66} \approx 0.00011 \\
K_3 &= 0.6366 \cdot e^{-15.46} \approx 1.24 \times 10^{-7} \\
K_4 &= 0.6366 \cdot e^{-3.86} \approx 0.0134 \\
K_5 &= K_2 \approx 0.00011
\end{aligned}$$

Density Estimation

An example:

- KDE estimate ($h = 0.5$):

$$\hat{p}_{h=0.5}(\mathbf{x}) = \frac{1}{5}(1.48\times10^{-5}+1.24\times10^{-7}+0.0134+2\times0.00011) \approx 0.00275$$

- Similarly, for $h = 1.0$, we found the KDE estimate:

$$\hat{p}_{h=1}(\mathbf{x}) = \frac{1}{5}(0.0111 + 0.0183 + 0.00334 + 0.0606 + 0.0183) \approx 0.0223$$

- And for $h = 2.0$, we found the KDE estimate:

$$\hat{p}_{h=2}(\mathbf{x}) = \frac{1}{5}(0.0204 + 0.0232 + 0.0151 + 0.0313 + 0.0232) \approx 0.0226$$

Density Estimation

An example:

- At small bandwidth $h = 0.5$, the query point is too far from all data, so density is very low.
- At larger bandwidth $h = 1.0$ or 2.0, the density becomes larger due to smooth spreading from nearby points (especially $\mathbf{x}_4$).

Density Estimation

Bandwidth h: an important paramter.

- Too small: The estimate is too "wiggly" (overfitting)
- Too large: The estimate is too smooth (underfitting)
- Can be chosen using methods such as cross-validation

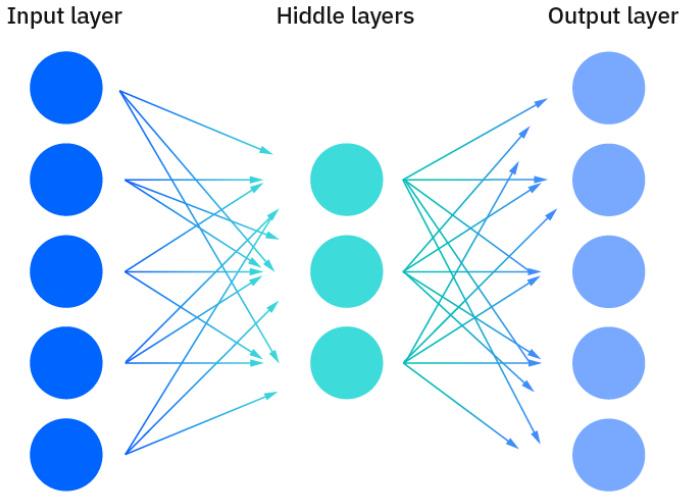
Outline

- Introduction: What is Unsupervised learning?
- K-Means Clustering
- Hierarchical Clustering
- Density Estimation
- Autoencoders for Dimensionality Reduction

Part III:
Autoencoders

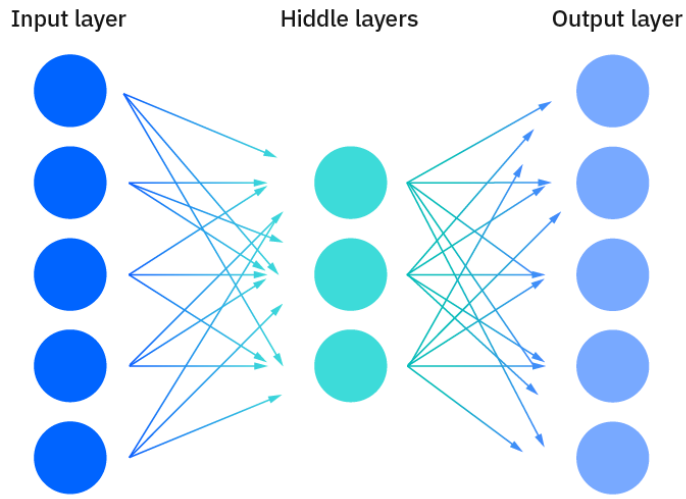
Autoencoders

- Autoencoders leverage neural networks to compress data and then recreate a new representation of the original data
- Encoding: from the input layer to the hidden layer
- Decoding: from the hidden layer to the output layer



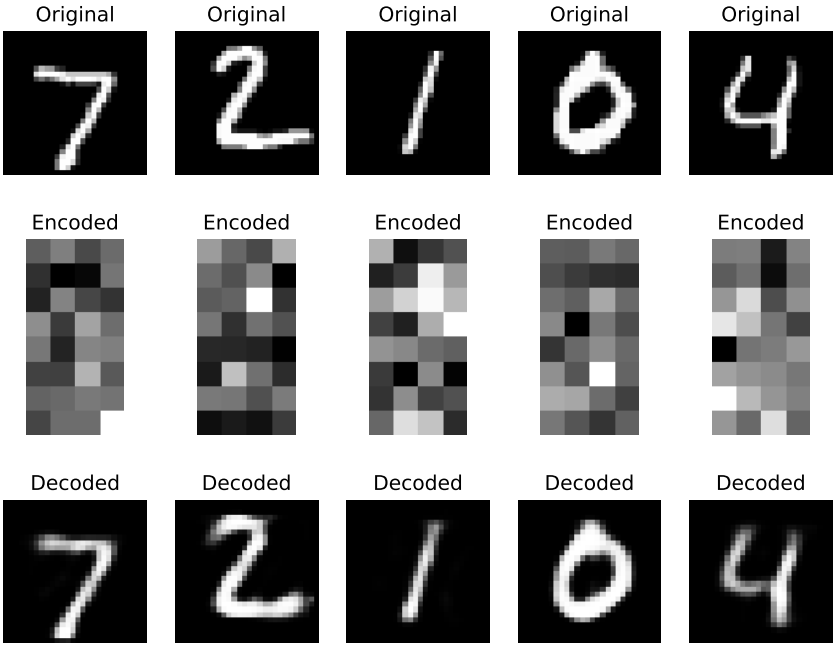
Autoencoders

- The hidden layer specifically acts as a bottleneck to compress the input layer prior to reconstructing within the output layer.



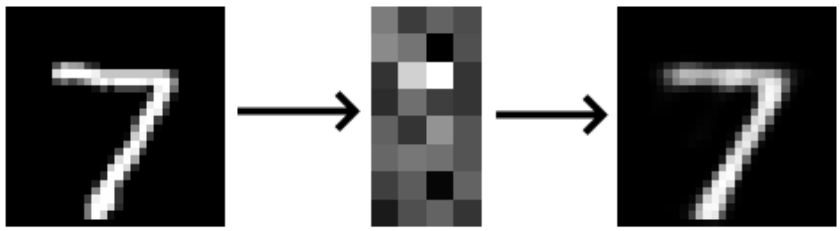
Autoencoders

An example: encoding 28×28 images into 8×4 images and decoding



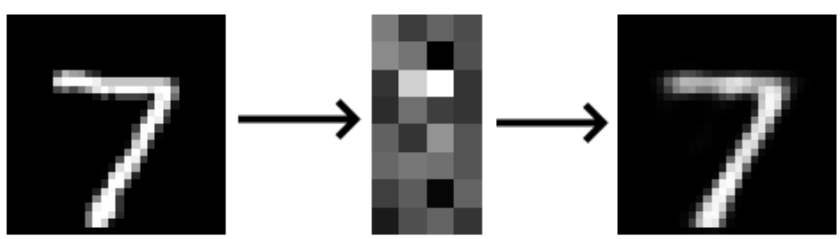
Autoencoders

- Assume that we have input data $\{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(n)}\}$ where $\mathbf{x}^{(i)} \in \mathbb{R}^d$
- We seek to learn an autoencoder that will output a new dataset $\{\mathbf{a}^{(1)}, \ldots, \mathbf{a}^{(n)}\}$ where $\mathbf{a}^{(i)} \in \mathbb{R}^k$ with $k < d$
- $\mathbf{a}^{(i)}$: a new representation of input data $\{\mathbf{x}^{(i)}\}$
- Example: encoded 28×28 images into 8×4 images ($d = 784$, $k = 32$)



Autoencoders

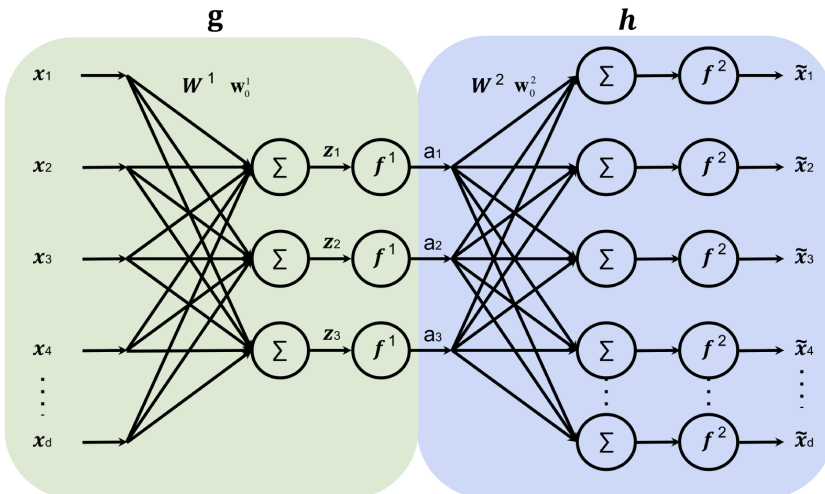
- An autoencoder consists of two functions:
  - A vector-valued encoder $g : \mathbb{R}^d \to \mathbb{R}^k$
  - A vector-valued decoder $h : \mathbb{R}^k \to \mathbb{R}^d$



- In general, the encoder and decoder functions might be any functions appropriate to the domain.
- Here, we are interested in encoders and decoders consist of neural networks

Autoencoders

- Here is a basic architecture of one such autoencoder, consisting of only one layer neural network in each of the encoder and decoder



- In this example, the original d-dimensional input is compressed into $k = 3$ dimensions

Autoencoders

- The encoder function is

$$g(\mathbf{x}; \mathbf{W}^1, \mathbf{w}_0^1) = f_1\left((\mathbf{W}^1)^{\mathrm{T}}\mathbf{x} + \mathbf{w}_0^1\right)$$

- Here $f_1(\cdot)$ is a nonlinear function applied to each dimension of the vector, and $\mathbf{W}^1 \in \mathbb{R}^{d\times k}$ and $\mathbf{w}_0^1 \in \mathbb{R}^k$
- Similarly, we have the function for the decoder

$$h(\mathbf{a}; \mathbf{W}^2, \mathbf{w}_0^2) = f_2\left((\mathbf{W}^2)^{\mathrm{T}}\mathbf{a} + \mathbf{w}_0^2\right)$$

- In general, the decoder and the encoder could involve multiple layers
- Goal: seeks parameters $\mathbf{W}^1, \mathbf{w}_0^1, \mathbf{W}^2, \mathbf{w}_0^2$ such that the reconstructed instances $\tilde{\mathbf{x}}$ are close to the original input $\mathbf{x}$

Autoencoder Learning

- To learn the weights, we use the same tools that we used for supervised learning, namely gradient descent method
- All we need is to specify the loss function $\mathcal{L}(\tilde{\mathbf{x}}, \mathbf{x})$ which measures the discrepancy between the reconstruction $\tilde{\mathbf{x}}$ and the original input
- For continuous-valued $\mathbf{x}$, we can use squared loss,

$$\mathcal{L}(\tilde{\mathbf{x}}, \mathbf{x}) = \sum_{j=1}^{d} \left(x_j - \tilde{x}_j\right)^2$$

- Then, we optimize the parameters of h and g so as to minimize the reconstruction error

$$\min \sum_{i=1}^{n} \mathcal{L}(\tilde{\mathbf{x}}^{(i)}, \mathbf{x}^{(i)})$$

Evaluating an Autoencoder

What makes a good autoencoder?

- Without further constraints, it is always possible to perfectly reconstruct the input.
- To learn something useful, we must create a bottleneck by making $k \ll d$.
- This forces the learning algorithm to seek transformations that describe the original data using as simple a description as possible
- As with clustering, autoencoders can be a preliminary step toward building other models, such as a regressor or classifier.